Belief Propagation for the Gaussian AR(1) Diffusion Model, Equation by Equation

From continuous messages to two-number updates, AMP, and the price of locality

Giovanni Mantovani
Bocconi University, MSc Data Science
working note following the call with J. Garnier-Brun, 5 June 2026

June 10, 2026

Abstract

This note does one thing slowly and completely: it derives belief propagation (BP) for the Gaussian AR(1) diffusion model *equation by equation*, assuming nothing as known. It is organised around the points raised in the call of 5 June 2026: (i) BP messages are probability distributions, and for continuous variables they are *functions* — numerically intractable to propagate as such; (ii) the practical resolution is to parametrise each message as a Gaussian and update two numbers, a mean and a variance — the construction known as AMP in computer science and TAP in physics; (iii) the proposed checkpoint: run BP with Gaussian messages on the model we can solve exactly, and check it recovers the known score; (iv) the proposed experiments: truncate the exact score matrix to a band, and separately cut the message iteration to a local range, and quantify the damage relative to the full computation; compare the Gaussian local-field approximation (AMP) against the exact score. We construct the factor graph from zero, write every sum–product update, compute every integral in display, and prove that for this model the Gaussian parametrisation is not an approximation: the update equations map Gaussians to Gaussians *exactly*, so two-number messages run BP without loss — the checkpoint passes at 10^{-14}. We then answer the call's open questions: the messages are exactly Gaussian here because every update is a linear-Gaussian operation; node-wise Gaussian messages do *not* in general imply a Gaussian joint law, but here the joint posterior is Gaussian for a separate reason (its log-density is a quadratic form); and the central-limit objection — "a CLT on two points is usually not right" — does not bite the BP mean (no averaging is invoked) but does bite the AMP *variance* closure, whose per-node update loses its fixed point beyond a measurable diffusion time. Experiments: truncating the score to tridiagonal costs up to $\sim 30\%$ relative RMS error at intermediate diffusion times and almost nothing at very small or very large times; cutting the message range to r hops behaves the same way but is strictly better than matrix banding at small times — truncated *inference* renormalises where truncated *matrices* do not. All numbers are produced by a self-contained, self-checking code (`bp_gaussian.py`) and an executed companion notebook.

Contents

1 What this note is, and where it comes from

This note is a from-scratch reconstruction. Its agenda is fixed by the call of 5 June 2026 with J. Garnier-Brun, and by nothing else. The points made in that call, in the order they will drive the document:

1. **The continuous-message problem.** BP messages are probability distributions. With discrete variables a message is a finite vector of probabilities — easy to store, easy to normalise, easy to update. With continuous variables a message is "theoretically a function over the entire support": the update rules become integrals, and the normalisation constant of every message would need to be recomputed at every step. Propagating functions numerically is the obstruction.
2. **The resolution to try.** Declare that the distribution being propagated is Gaussian, "and since it's Gaussian, then I can describe it with the mean and the variance … then what you do is that you update the mean and the variance." This two-number reduction is the step called *tapification*; the resulting algorithm family is AMP (approximate message passing) in computer science, the TAP equations in physics.
3. **The doubt.** The Gaussian form of a message is usually justified by a central limit theorem: at a factor node that aggregates *many* incoming messages, their sum is approximately Gaussian. Our factor graph is the opposite — "very tree-like, like a straight line with things on the side" — and every update takes essentially *two* inputs. "The central limit theorem on two data points is usually not right." Separately: even if the distribution on each node is Gaussian, it is "not entirely trivial" that the joint distribution of the whole sequence is Gaussian.
4. **The checkpoint.** "Go back to the case where a_{k+1} equals something times a_k plus a Gaussian. You knew how to solve it, you solved it. Can we actually do this with a message-passing algorithm? … the actual Gaussianity assumption on the BP messages, in the Gaussian data case … and see if we recover what we had."

5. **The experiments.** (a) "Imagine you take the exact score and, instead of using the full matrix, you truncate it to the tridiagonal thing — how bad is the approximation?" (b) With BP one can do better than truncating a matrix: "you restrain your factor graph … you cut the iteration at some point … the nodes are updated in such a way that we don't actually run through the entire graph, but we do local approximations every time. And then how bad is the inference compared to the real thing?" (c) The Gaussian local-field scheme (AMP) against the exact score.

6. **Why it matters.** The inference on every node requires propagating messages through the entire sequence — "clearly it's not local," which is the same observation as "the denoising matrix is not tridiagonal, it's actually full." But if, in some region of diffusion time, local inference is close to correct, "you could restrain the architecture very strongly … instead of having a fully connected model, [have] one that does patches, like a CNN."

Nothing in what follows is assumed known. Every object is defined before it is used; every equation is derived where it appears; every claim that can be checked numerically is checked, by the self-contained code in this folder.

2 The model, every symbol defined

The data. A *sequence* is a vector of K real numbers $a = (a_0, a_1, \ldots, a_{K-1}) \in \mathbb{R}^K$. We call k the *frame index*. The sequence is random, generated by the first-order autoregressive (AR(1)) rule

$$a_{k+1} \;=\; \alpha\, a_k \;+\; \eta_k, \qquad \eta_k \sim \mathcal{N}(0, \sigma_\eta^2) \text{ independent of everything}, \qquad |\alpha| < 1. \tag{1}$$

Here α is a fixed number measuring how strongly one frame remembers the previous one, and η_k is fresh Gaussian noise injected at every step. We start the chain at $a_0 \sim \mathcal{N}(0, \sigma_0^2)$.

Choice of constants. We choose

$$\sigma_0^2 = 1, \qquad \sigma_\eta^2 = 1 - \alpha^2. \tag{2}$$

This is the *stationary, unit-variance* normalisation, and it is a choice, not a necessity. Why it is convenient: if $\mathrm{Var}(a_k) = 1$, then by (1) and independence of η_k,

$$\mathrm{Var}(a_{k+1}) = \alpha^2 \,\mathrm{Var}(a_k) + \sigma_\eta^2 = \alpha^2 + (1 - \alpha^2) = 1,$$

so every frame has variance exactly 1: the chain looks statistically the same at every position, and no formula below needs to carry k-dependent variances.

The corruption (diffusion channel). Diffusion models destroy data with noise and learn to reverse the destruction. At *diffusion time* $t \geq 0$ (a second time axis, completely distinct from the frame index k), each frame is independently corrupted:

$$x_k \;=\; e^{-t}\, a_k \;+\; \xi_k, \qquad \xi_k \sim \mathcal{N}(0, D_t) \text{ i.i.d.}, \qquad D_t := 1 - e^{-2t}. \tag{3}$$

The number e^{-t} shrinks the signal; D_t is the variance of the added noise. The pair is matched so that $\mathrm{Var}(x_k) = e^{-2t} \cdot 1 + D_t = 1$ at every t: the channel is *variance-preserving*. At $t = 0$, $x_k = a_k$ (nothing happened); as $t \to \infty$, x_k becomes pure $\mathcal{N}(0, 1)$ noise (everything happened).

The object to compute. Write $P_t(x)$ for the probability density of the noisy sequence $x = (x_0, \dots, x_{K-1})$. The *joint score* is its log-gradient,

$$S(x,t) \;:=\; \nabla_x \log P_t(x) \;\in\; \mathbb{R}^K. \tag{4}$$

This is the function a diffusion model must learn: it is the drift that the reverse-time (generative) process adds to undo the noise, and the Bayes-optimal denoiser is an affine function of it (made exact in Step 3 below). "Joint" matters: S_k is the derivative of the log-density of the *whole* sequence with respect to x_k, and it will turn out to depend on every frame, not just on x_k.

3 The exact answer, derived from zero

Before any algorithm, we need the ground truth that the algorithms will be judged against. Everything in this section is elementary probability, written out in full.

Step 3.1: the clean covariance. Iterating (1) from frame j up to frame $i > j$,

$$a_i = \alpha^{\,i-j} a_j + \underbrace{\textstyle\sum_{l=j}^{i-1} \alpha^{\,i-1-l}\, \eta_l}_{\text{independent of } a_j}.$$

Multiplying by a_j and taking expectations (the noise terms are independent of a_j and have mean zero),

$$\mathbb{E}[a_i a_j] = \alpha^{\,i-j}\, \mathbb{E}[a_j^2] = \alpha^{\,i-j} \cdot 1.$$

By symmetry, for any i, j:

$$(\Sigma_0)_{ij} := \mathrm{Cov}(a_i, a_j) = \alpha^{|i-j|}. \tag{5}$$

The clean law is the centred Gaussian $P_0(a) = \mathcal{N}(a; 0, \Sigma_0)$ — Gaussian because a is a linear function of the i.i.d. Gaussian variables $(a_0, \eta_0, \eta_1, \dots)$.

Step 3.2: the noisy covariance and the exact score. Stack (3) as $x = e^{-t} a + \xi$ with $\xi \sim \mathcal{N}(0, D_t I)$ independent of a. Then x is again Gaussian and centred, with

$$\Sigma_t := \mathrm{Cov}(x) = e^{-2t}\, \Sigma_0 + D_t\, I. \tag{6}$$

The log-density of a centred Gaussian is $\log P_t(x) = -\frac{1}{2}\, x^\top \Sigma_t^{-1} x + \text{const}$, and for a symmetric matrix A one has $\nabla_x (x^\top A x) = 2Ax$ (differentiate the double sum $\sum_{ij} A_{ij} x_i x_j$ with respect to x_k: the terms containing x_k are $A_{kk} x_k^2$ and $\sum_{j \neq k} (A_{kj} + A_{jk}) x_k x_j$, giving $2 \sum_j A_{kj} x_j$). Hence

$$\boxed{\,S(x,t) = -\,\Sigma_t^{-1} x\,} \tag{7}$$

This is the benchmark: one $K \times K$ linear solve. The score is a *linear* function of x, and its coupling pattern is the matrix $Q_t := \Sigma_t^{-1}$, which is *dense* for $t > 0$ — this density is exactly the non-locality that the call's experiments are about.

Step 3.3: the score as a denoiser (Tweedie's identity). The same score can be computed without ever forming Σ_t, through the posterior over the clean sequence. Write the channel density

$$p_t(x \mid a) = \prod_{k=0}^{K-1} \frac{1}{\sqrt{2\pi D_t}} \exp\Big(-\frac{(x_k - e^{-t} a_k)^2}{2 D_t} \Big), \qquad P_t(x) = \int_{\mathbb{R}^K} P_0(a)\, p_t(x \mid a)\, \mathrm{d}a.$$

Differentiate P_t under the integral sign with respect to x_k (allowed: the integrand is smooth and dominated). Only the k-th channel factor depends on x_k, and

$$\partial_{x_k} p_t(x \mid a) = -\frac{x_k - e^{-t} a_k}{D_t}\, p_t(x \mid a).$$

Therefore

$$\partial_{x_k} P_t(x) = -\frac{1}{D_t} \int \big(x_k - e^{-t} a_k\big)\, P_0(a)\, p_t(x \mid a)\, \mathrm{d}a.$$

Now divide by $P_t(x)$ and recognise Bayes' rule: $P_0(a)\, p_t(x \mid a)/P_t(x) = p(a \mid x)$, the *posterior* density of the clean sequence given the noisy one. The ratio of integrals becomes a posterior expectation:

$$\boxed{S_k(x,t) = \partial_{x_k} \log P_t(x) = \frac{e^{-t}\, \mathbb{E}[a_k \mid x] \;-\; x_k}{D_t}} \tag{8}$$

Read it: the k-th score component is, up to known constants, the gap between the noisy frame and the channel-attenuated *best estimate of the clean frame given the whole noisy sequence*. Computing the score *is* computing the Bayesian denoiser $\mathbb{E}[a_k \mid x]$ — this is the form a message-passing algorithm will target, because $\mathbb{E}[a_k \mid x]$ is a per-node posterior mean, which is exactly what BP computes.

Step 3.4: the posterior, explicitly. We will need $p(a \mid x)$ itself. Up to factors that do not depend on a,

$$-\log p(a \mid x) = \tfrac{1}{2}\, a^\top Q_0\, a + \sum_{k=0}^{K-1} \frac{(x_k - e^{-t} a_k)^2}{2D_t} + \text{const}, \qquad Q_0 := \Sigma_0^{-1}.$$

Expanding the channel squares and collecting powers of a: each one contributes $\frac{e^{-2t}}{2D_t} a_k^2$ (quadratic) and $-\frac{e^{-t}}{D_t} x_k a_k$ (linear). Hence

$$p(a \mid x) \;\propto\; \exp\Big(-\tfrac{1}{2}\, a^\top J\, a + h^\top a\Big), \qquad J = \frac{e^{-2t}}{D_t} I + Q_0, \qquad h = \frac{e^{-t}}{D_t}\, x, \tag{9}$$

a Gaussian with mean $m = J^{-1} h$ and covariance J^{-1} (complete the square: $-\frac{1}{2} a^\top J a + h^\top a = -\frac{1}{2}(a-m)^\top J (a-m) + \text{const}$). Substituting m into (8) and simplifying recovers (7) — the code checks this identity at 10^{-15} rather than trusting the algebra.

Step 3.5: the one sparse object. The matrix $Q_0 = \Sigma_0^{-1}$ is *tridiagonal*. To see it without inverting anything, write the clean log-density using the chain rule of probability and the Markov property of (1):

$$P_0(a) = p(a_0) \prod_{k=0}^{K-2} p(a_{k+1} \mid a_k) = \mathcal{N}(a_0; 0, 1) \prod_{k=0}^{K-2} \mathcal{N}(a_{k+1}; \alpha a_k, \sigma_\eta^2),$$

$$-\log P_0(a) = \frac{a_0^2}{2} + \sum_{k=0}^{K-2} \frac{(a_{k+1} - \alpha a_k)^2}{2\sigma_\eta^2} + \text{const}.$$

This is a quadratic form whose matrix — which must equal Q_0, since the quadratic form of a Gaussian log-density is its precision — couples only adjacent indices: expanding $(a_{k+1} - \alpha a_k)^2$ produces a_k^2, a_{k+1}^2 and the cross term $-2\alpha\, a_k a_{k+1}$, never a product at distance two or more. Collecting coefficients,

$$(Q_0)_{kk} = \begin{cases} 1/\sigma_\eta^2, & k = 0 \text{ or } k = K-1, \\ (1+\alpha^2)/\sigma_\eta^2, & \text{otherwise}, \end{cases} \qquad (Q_0)_{k,k\pm1} = -\alpha/\sigma_\eta^2, \tag{10}$$

all other entries zero. (Boundary check at $k = 0$: the prior contributes $1 = (1-\alpha^2)/\sigma_\eta^2$ and the first transition contributes α^2/σ_η^2; the sum is $1/\sigma_\eta^2$.) The covariance is dense; the *precision* is sparse, and the sparsity is precisely the Markov property. The posterior precision J in (9) is Q_0 plus a multiple of the identity, hence also tridiagonal — the posterior, too, is a Markov chain. This is the structural fact the factor graph will draw.

4 The factor graph, constructed

Step 4.1: what a factor graph is. Suppose a density over variables $(a_0, \dots, a_{K-1})$ is a product of *factors*, each depending on a small subset of the variables: $p(a) \propto \prod_c \psi_c(a_{\partial c})$, where ∂c lists the variables factor ψ_c touches. The *factor graph* is the bipartite graph with one round node per variable, one square node per factor, and an edge between variable a_k and factor ψ_c exactly when $k \in \partial c$. It is a picture of "who talks to whom": all the algorithmic structure of BP reads off this picture.

Step 4.2: the factors of our posterior. The posterior (9) was assembled from three kinds of pieces, and we keep them separate because they play different roles in the algorithm:

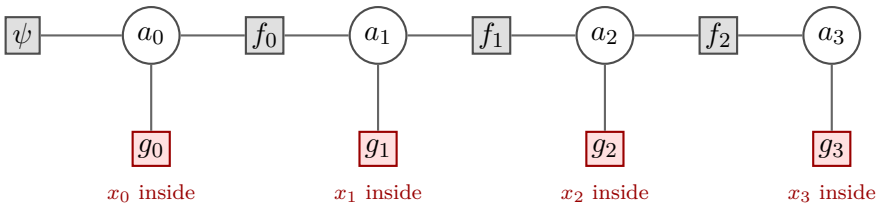
$$\begin{aligned} \text{prior factor:} \quad & \psi(a_0) = \mathcal{N}(a_0; 0, 1), \\ \text{transition factors:} \quad & f_k(a_k, a_{k+1}) = \mathcal{N}(a_{k+1}; \alpha a_k, \sigma_\eta^2), \qquad k = 0, \dots, K-2, \\ \text{observation factors:} \quad & g_k(a_k) = \mathcal{N}(x_k; e^{-t} a_k, D_t), \qquad k = 0, \dots, K-1, \end{aligned} \tag{11}$$

so that $p(a \mid x) \propto \psi(a_0) \prod_k f_k(a_k, a_{k+1}) \prod_k g_k(a_k)$. Two remarks that prevent standard confusions. First, the observed values x_k are *not* variable nodes: they are fixed numbers sitting inside the factors g_k, which are therefore functions of a_k alone. Second, each g_k, viewed as a function of a_k, is an unnormalised Gaussian *in* a_k: expanding the square in (11),

$$g_k(a_k) \;\propto\; \exp\Big(-\frac{e^{-2t}}{2D_t} a_k^2 + \frac{e^{-t}}{D_t} x_k a_k \Big) \;\propto\; \mathcal{N}\Big(a_k;\; \underbrace{e^t x_k}_{\text{de-attenuated obs.}},\; \underbrace{D_t e^{2t}}_{\text{its variance}} \Big). \tag{E}$$

We will call (E) the *pseudo-observation*: the data x_k, mapped back to the clean scale, with an uncertainty that grows with the noise level.

Step 4.3: the picture, and why it is a tree.



The graph for $K = 4$: a backbone of variables and transition factors, one observation leaf hanging off each variable, the prior capping the left end — "a straight line with things on the side," exactly the geometry described in the call. Counting: K variables and $2K$ factors (1 prior, $K-1$ transitions, K observations) make $3K$ nodes; the edges are $2(K-1)$ from transitions (each touches two variables), K from observations, 1 from the prior: $3K - 1$ edges in total. A connected graph with one fewer edge than nodes is a *tree* — no cycles. This single combinatorial fact is what will make BP exact: on a tree, information has exactly one route between any two nodes, so nothing can ever be counted twice.

5 Belief propagation, equation by equation

5.1 The general sum–product equations, and where they hurt

BP computes, for every variable, its marginal posterior — here $p(a_k \mid x)$, whose mean feeds (8). It does so by passing *messages* along the edges of the factor graph. There are two kinds.

A **variable-to-factor** message: variable a_i tells factor ψ_c what it currently believes about itself, based on everyone *except* ψ_c:

$$\nu_{i\to c}(a_i) \;\propto\; \prod_{b\,\in\,\partial i\setminus c} \hat{\nu}_{b\to i}(a_i). \tag{SP1}$$

Here ∂i is the set of factors touching a_i, and the product runs over all of them *except the recipient* c — the recipient must not hear its own previous opinion echoed back, or information would circulate and inflate.

A **factor-to-variable** message: factor ψ_c tells variable a_i what the factor implies about it, after averaging over the factor's *other* arguments, each weighted by what that argument currently believes:

$$\hat{\nu}_{c\to i}(a_i) \;\propto\; \int \psi_c(a_{\partial c}) \prod_{j\,\in\,\partial c\setminus i} \nu_{j\to c}(a_j)\, \mathrm{d}a_{\partial c\setminus i}. \tag{SP2}$$

The **belief** (the marginal estimate) at variable a_i multiplies everything that arrives:

$$b_i(a_i) \;\propto\; \prod_{c\,\in\,\partial i} \hat{\nu}_{c\to i}(a_i). \tag{SP3}$$

On a tree, the beliefs are *exactly* the true marginals once each edge has carried one message in each direction.

Now the difficulty named in the call. Every message above is a probability distribution in its argument, defined "up to proportionality" — meaning it must be normalised. If a_i takes q discrete values, a message is a vector of q numbers; (SP1) is an entrywise product, (SP2) a finite sum, normalisation a division by the total. If a_i is *continuous*, a message is a *function* on $\mathbb{R}$: (SP2) is an integral transform, and the normalising constant is itself an integral that changes at every update. Propagating arbitrary functions and renormalising them numerically, at every edge and every step, is the intractability that motivated the call.

5.2 The two facts that rescue the continuous case here

For *this* model every factor in (11) is Gaussian, and two elementary facts about Gaussians close the algebra. We state and prove both; together they will guarantee that every message in (SP1)–(SP3) is an (unnormalised) Gaussian, hence carried by two numbers — and the normalisation problem evaporates, because a Gaussian normalises itself: its constant is determined by its (mean, variance) pair and never needs to be tracked.

Lemma 1 (products)**.** *For any means m_1, m_2 and variances v_1, v_2,*

$$\mathcal{N}(a; m_1, v_1)\cdot\mathcal{N}(a; m_2, v_2) \;\propto\; \mathcal{N}(a; m, v), \qquad \frac{1}{v} = \frac{1}{v_1} + \frac{1}{v_2}, \qquad \frac{m}{v} = \frac{m_1}{v_1} + \frac{m_2}{v_2}.$$

Proof. Write each factor as $\exp(-\frac{a^2}{2v_i} + \frac{m_i}{v_i}a + \text{const})$. The product adds the exponents, so the coefficient of $-a^2/2$ is $1/v_1 + 1/v_2$ and the coefficient of a is $m_1/v_1 + m_2/v_2$. Any function of the form $\exp(-\lambda a^2/2 + \theta a)$ is proportional to $\mathcal{N}(a; \theta/\lambda, 1/\lambda)$. □

In words: *precisions add; precision-weighted means add.* This is the engine of every "combine" step below.

Lemma 2 (linear propagation)**.** *For any $\alpha \neq 0$, $s^2 > 0$:*

$$\textit{(forward)} \quad \int \mathcal{N}(a';\alpha a, s^2)\,\mathcal{N}(a;m,v)\,\mathrm{d}a = \mathcal{N}(a';\alpha m,\ \alpha^2 v + s^2),$$

$$\textit{(backward)} \quad \int \mathcal{N}(a';\alpha a, s^2)\,\mathcal{N}(a';m,v)\,\mathrm{d}a' \ \propto\ \mathcal{N}\Big(a;\ \frac{m}{\alpha},\ \frac{v+s^2}{\alpha^2}\Big).$$

Proof. Forward: the integral is the density of $a' = \alpha a + \eta$ where $a \sim \mathcal{N}(m, v)$ and $\eta \sim \mathcal{N}(0, s^2)$ are independent; a linear combination of independent Gaussians is Gaussian with mean αm and variance $\alpha^2 v + s^2$. Backward: viewed as a function of a, the first factor satisfies $\mathcal{N}(a';\alpha a, s^2) \propto \mathcal{N}(a; a'/\alpha, s^2/\alpha^2)$ (divide the exponent's argument by α), so the integral is the forward case run through the inverted map $a = (a' - \eta)/\alpha$, giving mean m/α and variance $(v + s^2)/\alpha^2$. □

5.3 Specialising the equations to the chain

We now instantiate (SP1)–(SP3) on the factor graph of Step 4.3, edge by edge, and watch most of them collapse.

Step 5.1: messages from leaf factors are just the factors. The prior ψ and each observation g_k touch a single variable, so in (SP2) there is nothing to integrate and no incoming message to weight: $\hat{\nu}_{\psi\to a_0}(a_0) = \psi(a_0) = \mathcal{N}(a_0; 0, 1)$ and $\hat{\nu}_{g_k\to a_k}(a_k) = g_k(a_k)$, the pseudo-observation (E). Leaves simply inject their factor.

Step 5.2: messages along the backbone, and the bookkeeping. Consider the edge from variable a_k into the transition factor f_k (to its right). Rule (SP1) says: multiply everything arriving at a_k except from f_k — that is, the message from the left transition f_{k-1} (or the prior, if $k = 0$) and the message from the observation g_k:

$$\nu_{a_k\to f_k}(a_k) \ \propto\ \hat{\nu}_{f_{k-1}\to a_k}(a_k)\cdot g_k(a_k). \tag{12}$$

Then rule (SP2) at f_k, whose other argument is a_{k+1}:

$$\hat{\nu}_{f_k\to a_{k+1}}(a_{k+1}) = \int f_k(a_k, a_{k+1})\,\nu_{a_k\to f_k}(a_k)\,\mathrm{d}a_k. \tag{13}$$

Define the *forward message* $\mu_{\to k} := \hat{\nu}_{f_{k-1}\to a_k}$ (with $\mu_{\to 0} := \hat{\nu}_{\psi\to a_0}$) and substitute (12) into (13). The two-rule dance collapses into a single recursion, which we now write with every operation labelled:

$$\mu_{\to 0}(a_0) = \mathcal{N}(a_0; 0, 1) \qquad \text{(start: the prior)} \qquad \text{(F0)}$$

$$\text{combine: } (m_k^c, v_k^c) = \text{Lemma 1}\big[\mu_{\to k}\,,\ g_k\big] \qquad \text{(absorb the local data } x_k\text{)} \qquad \text{(F1)}$$

$$\mu_{\to k+1}(a_{k+1}) = \mathcal{N}\big(a_{k+1};\ \alpha m_k^c,\ \alpha^2 v_k^c + \sigma_\eta^2\big) \qquad \text{(Lem. 2, fwd)} \qquad \text{(F2)}$$

Spelled out, (F1) reads: with the pseudo-observation (E),

$$\frac{1}{v_k^c} = \frac{1}{v_{\to k}} + \frac{e^{-2t}}{D_t}, \qquad \frac{m_k^c}{v_k^c} = \frac{m_{\to k}}{v_{\to k}} + \frac{e^{-t}}{D_t}x_k.$$

By symmetry, peeling the chain from the right gives the *backward message* $\mu_{\leftarrow k} := \hat{\nu}_{f_k\to a_k}$:

$$\mu_{\leftarrow K-1}(a_{K-1}) \equiv 1 \qquad \text{(start: nothing right of the end — flat)} \qquad \text{(B0)}$$

$$\text{combine: } (m_k^c, v_k^c) = \text{Lemma 1}\big[\mu_{\leftarrow k}\,,\ g_k\big] \qquad \text{(absorb } x_k\text{)} \qquad \text{(B1)}$$

$$\mu_{\leftarrow k-1}(a_{k-1}) = \mathcal{N}\Big(a_{k-1};\ \tfrac{m_k^c}{\alpha},\ \tfrac{v_k^c+\sigma_\eta^2}{\alpha^2}\Big) \qquad \text{(Lem. 2, bwd)} \qquad \text{(B2)}$$

The flat start (B0) is the $v \to \infty$ limit of a Gaussian (precision 0); Lemma 1 handles it exactly — multiplying by a flat message changes nothing.

Finally the belief (SP3) at node k multiplies the three arrivals — left transition, observation, right transition:

$$p(a_k \mid x) \;\propto\; \mu_{\to k}(a_k) \;\cdot\; g_k(a_k) \;\cdot\; \mu_{\leftarrow k}(a_k), \tag{C}$$

two applications of Lemma 1. Its mean is $\mathbb{E}[a_k \mid x]$, which enters Tweedie (8) to give the score.

Remark 1 (the one trap, stated loudly). In (F1) the local observation g_k is absorbed *when leaving* node k, so the forward message $\mu_{\to k}$ *arriving* at node k contains the data $x_0, \ldots, x_{k-1}$ and **not** x_k; symmetrically $\mu_{\leftarrow k}$ contains $x_{k+1}, \ldots, x_{K-1}$ and not x_k. The combination (C) then counts x_k exactly once — via its own factor g_k. The tempting shortcut of folding g_k *into* the stored message $\mu_{\to k}$ makes (C) count x_k twice and silently corrupts every marginal. This bookkeeping is forced by the $\backslash c$ exclusions in (SP1)–(SP2); it is the single most error-prone point of an implementation, which is why the code states it in its header comment and the checks of §5.5 would catch the violation at order one.

Step 5.3: the algorithm, assembled.

> **Gaussian BP on the AR(1) diffusion chain.** Input: $x \in \mathbb{R}^K$, α, t.
> 1. **Forward sweep** (left to right): start with (F0); for $k = 0, \ldots, K-2$ apply (F1) then (F2). Store $(m_{\to k}, v_{\to k})$ for every k.
> 2. **Backward sweep** (right to left): start with (B0); for $k = K-1, \ldots, 1$ apply (B1) then (B2). Store $(m_{\leftarrow k}, v_{\leftarrow k})$.
> 3. **Combine** at every k with (C): posterior mean m_k and variance v_k.
> 4. **Score** by Tweedie (8): $S_k = (e^{-t} m_k - x_k)/D_t$.
>
> Cost: each sweep is $K-1$ constant-cost updates on number pairs; total $O(K)$, against the $O(K^3)$ of solving (7) naively.

5.4 Are the messages Gaussian? Are we approximating?

Claim 1. *On this model, every message and every belief produced by (F0)–(C) is exactly Gaussian (or flat), at every node and every (K, α, t). The two-number parametrisation is therefore lossless: Gaussian BP here is BP, with no approximation anywhere.*

Proof. Induction along each sweep. Base cases: (F0) is Gaussian; (B0) is the flat limit. Induction step: (F1)/(B1) are products of two Gaussians (Lemma 1: Gaussian); (F2)/(B2) are linear-Gaussian integrals (Lemma 2: Gaussian). The combination (C) is a double product (Gaussian). No step ever leaves the family. □

This settles the call's first question *for this model*, and the mechanism is worth stating because it transfers: the closure holds because every operation the graph demands — multiply two Gaussians in the same variable, push a Gaussian through a *linear* transition with Gaussian noise — is an operation under which the Gaussian family is invariant. It is an *algebraic* property of the factors, valid at any node degree, including degree two. No central limit theorem was invoked, so there is no "CLT on two points" to fail. (The criterion proposed in the call is exactly right: the updates here are linear in the propagated statistics — had the transition been nonlinear, $a_{k+1} = \phi(a_k) + \eta_k$, Lemma 2 would fail and the messages would leave the Gaussian family; *that* is the case where Gaussian parametrisation becomes a genuine approximation.)

Question 1. If the message at each node is Gaussian, is the joint distribution of the whole sequence Gaussian? (Raised in the call: "it's not entirely trivial … you've updated everyone with everyone else.")

Answer. The implication is false in general, and the doubt is correct: marginal Gaussianity does not imply joint Gaussianity (a standard counterexample couples two standard Gaussians through a non-Gaussian copula — both marginals are exactly $\mathcal{N}(0,1)$, the pair is not jointly Gaussian). BP messages and beliefs are *per-node marginal* objects; nothing about their shape constrains the joint law. For *our* posterior the joint *is* Gaussian, but for a reason independent of message passing: the log-density (9) is a quadratic form in a, globally. The correct logic is therefore: joint Gaussianity comes from the model (quadratic log-density); message Gaussianity then follows from it (any marginal or conditional of a joint Gaussian is Gaussian); never the reverse.

5.5 The checkpoint: BP recovers the exact score

The call's proposed test: run the message passing on the solvable case and compare with the known answer. We run it on a worked instance small enough to print every number, then sweep the parameter space.

A complete instance. $K = 4$, $\alpha = 0.6$, $t = 0.3$, $x = (0.9, -0.2, 0.5, -1.1)$, so $e^{-t} = 0.740818$, $D_t = 0.451188$, $\sigma_\eta^2 = 0.64$, pseudo-observation variance $D_t e^{2t} = 0.822119$. Running (F0)–(C):

	$k=0$	$k=1$	$k=2$	$k=3$
forward $\mu_{\to k}$ (m, v)	(0, 1)	(0.40004, 0.80243)	(0.04146, 0.78619)	(0.21067, 0.78468)
pseudo-obs. (E) (m, v)	(1.21487, 0.82212)	(−0.26997, 0.82212)	(0.67493, 0.82212)	(−1.48485, 0.82212)
backward $\mu_{\leftarrow k}$ (m, v)	(−0.29429, 3.64415)	(0.24117, 3.67700)	(−2.47474, 4.06144)	flat
posterior (mean, var)	(0.56086, 0.40148)	(0.08621, 0.36569)	(0.09668, 0.36569)	(−0.61733, 0.40148)

One step verified by hand so the table is auditable. Forward into $k = 1$: combine (F0) with the pseudo-observation at $k = 0$ (Lemma 1): precision $1 + 1/0.82212 = 2.21637$, variance 0.45119, mean $0.45119 \times (0 + 1.21487/0.82212) = 0.66673$; then (F2): mean $0.6 \times 0.66673 = 0.40004$, variance $0.36 \times 0.45119 + 0.64 = 0.80243$. Matches the table. The Tweedie line (8) then gives

$$S^{\mathrm{BP}}(x,t) = (-1.07384,\ 0.58482,\ -0.94945,\ 1.42439),$$

and the matrix benchmark (7) gives the same four numbers to all printed digits. Note also the symmetry $v_0 = v_3$ and $v_1 = v_2$ in the posterior variances — variances do not depend on the data x, only on the geometry, and the chain is symmetric under reversal.

The sweep. Over $K \in \{2,3,6,12\}$, $\alpha \in \{0.3, 0.7, 0.9, -0.5\}$, $t \in \{0.05, 0.4, 1.2, 3\}$ with random x (64 configurations), the largest deviation between the BP pipeline and the exact computation is

$$\max \left| m_k^{\mathrm{BP}} - (J^{-1}h)_k \right| = 2.0 \times 10^{-15}, \qquad \max \left| v_k^{\mathrm{BP}} - (J^{-1})_{kk} \right| = 2.4 \times 10^{-15},$$
$$\max \left| S_k^{\mathrm{BP}} - S_k \right| = 9.8 \times 10^{-15}.$$

This is double-precision rounding. **The checkpoint passes**: the message-passing algorithm with two-number Gaussian messages reproduces the solved case exactly, as conjectured in the call — and Claim 1 explains why the agreement is exact rather than approximate.

6 AMP: the per-node Gaussian field, and what it costs

6.1 From per-edge messages to per-node statistics

The call's description of the general recipe: when the BP updates are intractable, parametrise the local distributions as Gaussians and "reduce everything to a mean and variance update." The fully reduced scheme — AMP / TAP — goes one step further than §5.3: instead of a (mean, variance) pair *per directed edge* (our $\mu_{\to k}$, $\mu_{\leftarrow k}$), it keeps a single pair (m_i, V_i) *per node* and updates it against the aggregate of all neighbours. Working on the posterior in natural parameters (9) — precision J (tridiagonal), field h — the per-node updates read

$$m_i \;\leftarrow\; \frac{1}{J_{ii}}\Big(h_i - \sum_{j\neq i} J_{ij}\, m_j\Big), \tag{A1}$$

$$V_i \;\leftarrow\; \frac{1}{J_{ii} - \sum_{j\neq i} J_{ij}^2\, V_j}. \tag{A2}$$

Where these come from, stated without mystery: node i models the combined effect of its neighbours as a Gaussian field. For the *mean*, the stationarity condition of (A1) is literally the i-th row of $Jm = h$. For the *variance*, each neighbour j contributes its current uncertainty V_j propagated through the coupling J_{ij} (squared, because variances add under linear maps), and (A2) is the self-consistent solution — with the crucial simplification that node i uses the neighbour's *full* variance V_j, not the variance the neighbour would have *if i were absent* (the per-edge, "cavity" quantity that true BP keeps). Dropping that exclusion is exactly what makes the scheme per-node instead of per-edge; it is harmless when each node has many weak neighbours — removing one of N changes little, and the aggregated field is near-Gaussian by the central limit theorem, with the $1/\sqrt{N}$ convergence noted in the call — and it is an order-one modification when the neighbours number two.

6.2 What survives: the score

Claim 2. *Any fixed point of* (A1) *satisfies $Jm = h$ exactly — the same linear system whose solution is the true posterior mean. The variance update* (A2) *does not enter* (A1) *at all. Hence AMP's mean, wherever its iteration converges, equals the exact posterior mean, and by Tweedie* (8) ***AMP's score equals the exact score****, at every diffusion time — regardless of how wrong, or even nonexistent, its variances are.*

Measured (chain, $K = 12$, $\alpha = 0.8$; damped iteration of (A1)): the gap between the AMP mean and the exact mean is below 3×10^{-12} at every tested t. For the *score* — the object diffusion needs — the Gaussian local-field approximation on this model is not an approximation at all.

6.3 What breaks: the variance

The variance closure (A2) is genuinely different from BP's, and on a two-neighbour chain the difference is visible in one line. In the homogeneous interior of a long chain, J has constant diagonal $J_{\mathrm{d}} = e^{-2t}/D_t + (1+\alpha^2)/\sigma_\eta^2$ and constant off-diagonal $\beta = -\alpha/\sigma_\eta^2$. BP's per-edge variance bookkeeping — one neighbour excluded — has fixed points solving

$$\lambda = J_{\mathrm{d}} - \frac{\beta^2}{\lambda} \quad\Longleftrightarrow\quad \lambda^2 - J_{\mathrm{d}}\lambda + \beta^2 = 0,$$

with real solutions iff $J_{\mathrm{d}}^2 \geq 4\beta^2$ — which always holds, since $J_{\mathrm{d}} - 2|\beta| = e^{-2t}/D_t + (1 - |\alpha|)^2/\sigma_\eta^2 > 0$. The per-node closure (A2) — both neighbours included — has fixed points

solving

$$V = \frac{1}{J_\mathrm{d} - 2\beta^2 V} \quad \Longleftrightarrow \quad 2\beta^2 V^2 - J_\mathrm{d} V + 1 = 0,$$

with real solutions iff $J_\mathrm{d}^2 \geq 8\beta^2$. The entire difference between BP and AMP on this chain is the factor 2 — the dropped exclusion — and it moves the existence threshold from "always" to a condition that *fails* once the prior coupling dominates the evidence, i.e. at large enough t for strong enough α. Beyond that point the bracket in (A2) turns negative during the iteration: there is no positive fixed point, and the honest report is failure, not a clipped number. Measured breakdown times of the iteration ($K = 200$, bisection):

$$\alpha = 0.5 : \ t \approx 0.86, \qquad \alpha = 0.7 : \ t \approx 0.36, \qquad \alpha = 0.9 : \ t \approx 0.11,$$
$$\alpha = 0.3 : \ \text{no breakdown found up to } t = 30.$$

This is the precise content of the call's objection: the central limit argument that justifies treating the aggregated neighbour field as an independent Gaussian is a many-neighbour argument, and on two neighbours its failure is not a small distortion — beyond a threshold the self-consistent variance simply ceases to exist. But note exactly *where* the objection lands: on the variance (A2), never on the mean (A1), whose fixed-point equation is closure-independent. The "CLT on two points" worry is correct, and the score survives it.

7 Experiment 1: how bad is locality?

The exact score matrix $Q_t = \Sigma_t^{-1}$ is full for every $t > 0$: inference at one frame draws on all frames. The call proposed to quantify the cost of pretending otherwise, in two distinct ways that we keep carefully apart.

Variant 1a — truncate the matrix. Replace Q_t by $Q_t^{(b)}$, keeping only entries with $|i-j| \leq b$ ($b = 1$ is the tridiagonal truncation suggested in the call), and use $\hat{S}(x) = -Q_t^{(b)} x$.

Variant 1b — truncate the inference. "Cut the iteration": let messages travel at most r hops, replacing any message from beyond the cut by its no-evidence value — the stationary prior. Concretely, the estimate of a_k uses exact inference on the window $x_{k-r}, \ldots, x_{k+r}$ alone (a contiguous block of the stationary chain is itself a stationary AR(1) chain, so the windowed problem is again of the same form); the score follows by Tweedie.

The error measure, with no randomness. Every estimator above is *linear* in x: $\hat{S}(x) = -Mx$ for a computable matrix M, while the truth is $S(x) = -Q_t x$. Averaging over the data distribution $x \sim P_t = \mathcal{N}(0, \Sigma_t)$, the mean squared error is a trace (using $\mathbb{E}[x^\top A x] = \mathrm{tr}(A\Sigma_t)$ for any matrix A):

$$\mathbb{E}\big\|\hat{S}(x) - S(x)\big\|^2 = \mathrm{tr}\big((M - Q_t)\,\Sigma_t\,(M - Q_t)^\top\big), \qquad \mathbb{E}\|S(x)\|^2 = \mathrm{tr}\big(Q_t \Sigma_t Q_t\big) = \mathrm{tr}\, Q_t,$$

and we report the *relative root error* — the square root of the ratio. No samples, no seeds: each point in the figures is a deterministic number.

Findings. (Figures 1–2; selected values for $b = r = 1$, $K = 40$, $\alpha = 0.8$.)

relative RMS error	$t = 0.05$	$t = 0.3$	$t = 1$	$t = 3$
1a: tridiagonal matrix ($b = 1$)	0.210	0.302	0.135	0.0035
1b: message range $r = 1$	0.123	0.235	0.130	0.0035

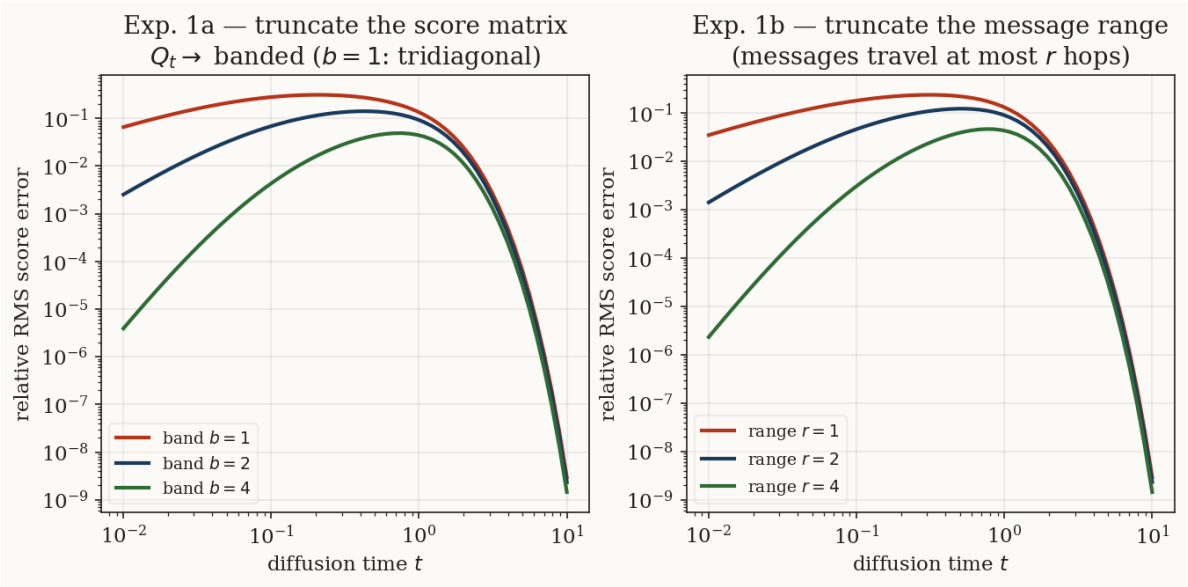


Figure 1: Relative RMS score error of the two truncations ($K = 40$, $\alpha = 0.8$) as a function of diffusion time. Left: banded score matrix, $b \in \{1, 2, 4\}$. Right: truncated message range, $r \in \{1, 2, 4\}$.

1. **The cost of locality is non-monotone in t and peaks at intermediate times.** At $t \to 0$ the score is genuinely near-tridiagonal (the posterior precision is Q_0 plus a large diagonal), and at $t \to \infty$ the score decouples entirely ($Q_t \to I$, $S \to -x$); in between, the truncated estimator misses up to $\sim 30\%$ of the score in RMS. This confirms quantitatively the intuition discussed in the call: the matrix "starts tridiagonal, becomes full at intermediate times" — and intermediate times are where locality genuinely hurts.
2. **Truncated inference beats the truncated matrix at small t, at equal locality.** At $t = 0.05$: 12.3% versus 21.0%. The reason is worth recording: variant 1b solves a *correctly normalised* inference problem on the window (the window's own prior block), while variant 1a zeroes entries of an inverse, which is not the inverse of anything and in particular misstates the diagonal. Cutting the *algorithm* is better than cutting the *answer*. The two coincide at large t, where all off-diagonal content dies anyway.
3. **The error decays geometrically in the allowed range** (Fig. 2), with a rate that worsens as t grows toward the intermediate regime: each extra hop of message range buys a fixed multiplicative factor of accuracy, and the factor is closest to 1 precisely where the band of Q_t is fullest.
4. **Architecture reading** (the call's motivation): a score network with a restrained, local receptive field — patches, a CNN — is near-optimal at small and large diffusion times, and pays a quantifiable price (here up to $\sim 30\%$ RMS at range 1, $\sim 13\%$ at range 2, $\sim 5\%$ at range 4) in the mid-diffusion window. If the architecture's range can grow with t — or simply be sized for the worst time — the locality of the underlying Markov data is exploitable essentially for free.

8 Experiment 2: AMP against the exact score

The second comparison asked in the call: the Gaussian local-field approximation against the truth, on the chain. We iterate (A1) and (A2) (damping 0.5) and compare with the exact posterior mean and variances; Fig. 3 and the table give the result ($K = 12$, $\alpha = 0.8$).

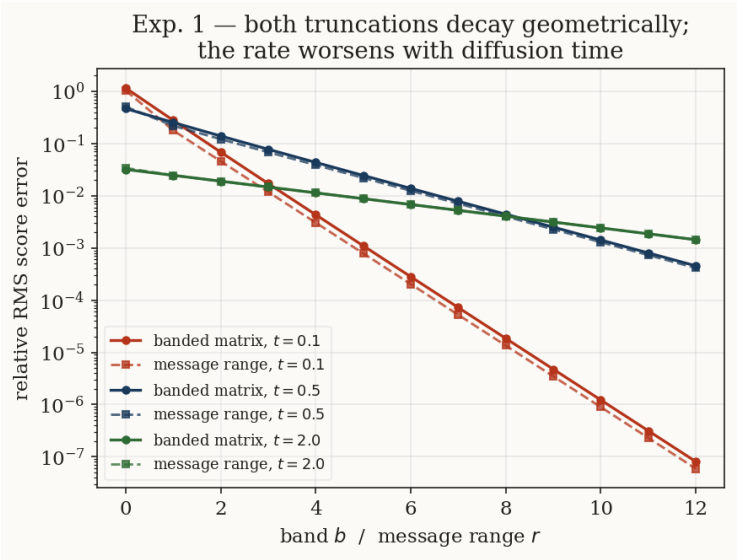


Figure 2: The same errors as a function of the band / range at three fixed times. Both truncations converge geometrically as the allowed range grows; the rate is fast at $t = 0.1$, slowest near $t \sim 0.5$.

t	mean (= score) error	variance error
0.05	1.8×10^{-13}	1.3×10^{-4}
0.2	4.7×10^{-13}	3.5×10^{-2}
0.5	1.2×10^{-12}	no fixed point
1.0	2.6×10^{-12}	no fixed point

Verdict. The two halves of the call's intuition are both confirmed, and they separate cleanly:

- the conjecture — run the Gaussian message passing on the solved case and "recover what we had" — holds *exactly* for the score, both for true BP (§5.5, where it holds by algebraic closure) and for AMP (Claim 2, where it holds because the mean update's fixed point is closure-independent);
- the objection — "a CLT on two data points is usually not right" — is also exactly right, and its target is the *variance* closure (A2): accurate while the per-frame evidence dominates (small t), degrading as the prior coupling takes over, and ceasing to exist past a threshold time that shrinks as the chain coupling α grows (§6.3).

On this model one can therefore use the cheap per-node scheme for the score with impunity, but must not trust — past small t, must not even expect to compute — its uncertainties. Any application needing calibrated posterior variances (likelihoods, error bars, exploration) needs the per-edge messages of §5.3, which cost the same $O(K)$ and are exact.

9 Summary, and what to bring to the next call

Established here.

1. The continuous-message obstruction dissolves for this model: every sum–product message is exactly Gaussian (Claim 1), because every update is one of two operations under which the Gaussian family is closed. Two numbers per message; normalisation never tracked.

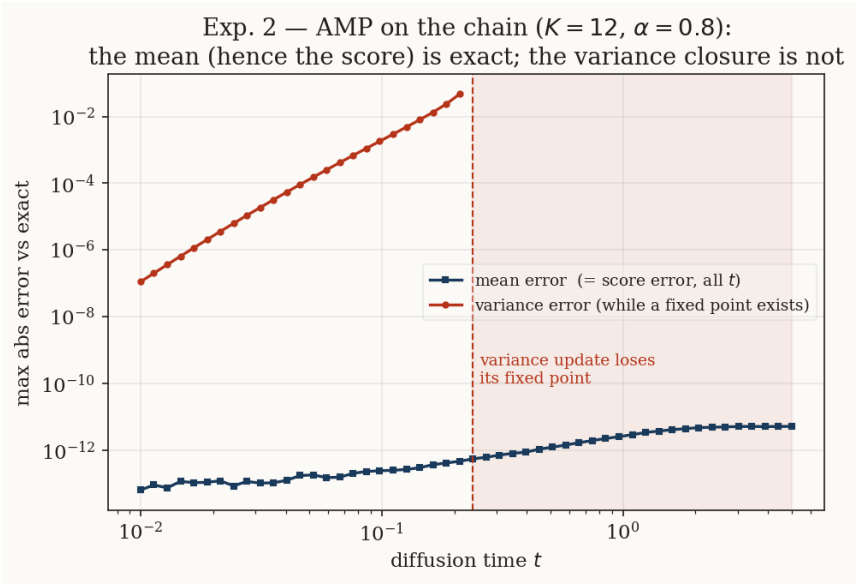


Figure 3: AMP on the chain. The mean error (navy) sits at numerical noise for every diffusion time — the score is exact, Claim 2. The variance error (red) grows steadily with t and, past $t \approx 0.23$, the update (A2) has no positive fixed point at all (shaded).

2. The checkpoint passes: Gaussian BP returns the known score to 10^{-14}, in $O(K)$, with every message derived ((F0)–(C)) and printed on a worked instance.
3. Node-wise Gaussianity does not imply joint Gaussianity in general; here the joint posterior is Gaussian because its log-density is quadratic — the implication runs from the model to the messages, not conversely.
4. AMP (per-node mean/variance) returns the *same score* at every t — the mean fixed point is $Jm = h$ regardless of the variance closure — while its variance is accurate only at small t and has no fixed point beyond a measurable threshold: the "CLT on two points" objection, located precisely.
5. Locality costs at most $\sim 30\%$ relative RMS (tridiagonal, worst t) and almost nothing at small or large t; truncating the *inference* is uniformly no worse, and at small t clearly better, than truncating the *matrix*.

Next steps suggested by the call, in order of leverage.

1. **The discrete-data route.** Keep the Markov chain on a finite alphabet (a transition matrix, no continuous innovation), add the Gaussian channel only at diffusion. Messages become finite vectors, BP is exact and trivially implementable, and the Bayes-optimal denoiser is computable — a second fully solvable pole, with non-Gaussian data, against which the Gaussian-approximation questions can be asked sharply.
2. **Non-Gaussian innovations (e.g. Laplace).** Lemma 2 fails; messages leave the Gaussian family; projecting them back (moment matching) makes Gaussian BP a genuine approximation. The call's expectation — the quality "will strongly depend on the data distribution" — becomes testable on the same factor graph, with the same two experiments.
3. **Time-adaptive locality.** Experiment 1 gives, for each t, the smallest range achieving a target error: a principled receptive-field schedule for a restrained score architecture (the

call's CNN/patches remark), to be compared against what trained local networks actually learn.

Reproducibility. Everything in this note is generated by two files in this folder: `bp_gaussian.py` (model, exact score, BP, truncations, AMP, error metrics — with built-in checks; run it) and `experiments.py` (the figures and every number quoted). The executed notebook `companion.ipynb` walks the same path interactively. The bibliography is deliberately short: the two references named in the call.

References

[1] M. Mézard and A. Montanari, *Information, Physics, and Computation*, Oxford University Press, 2009. Chapter 14 (belief propagation; the sum–product equations 14.14–14.15 discussed in the call).

[2] L. Zdeborová and F. Krzakala, *Statistical Physics for Optimization and Learning*, EPFL doctoral lecture notes, 2021. (The Gaussian-parametrised message passing — mean and variance updates — and the $1/\sqrt{N}$ central-limit control of the aggregated field discussed in the call.)